

# Digitale Forensik

Zusammenfassung

Luis Staudt

## Inhaltsverzeichnis

|                                                       |    |
|-------------------------------------------------------|----|
| Inhaltsverzeichnis . . . . .                          | 2  |
| 1 Grundlagen . . . . .                                | 5  |
| 1.1 Allocated vs. Unallocated Space . . . . .         | 5  |
| 1.2 EWF-Format (E01) . . . . .                        | 5  |
| 1.3 RAW → EWF → xmount Kreislauf . . . . .            | 5  |
| 1.4 Physische vs. logische Analyse . . . . .          | 6  |
| 1.5 File Carving . . . . .                            | 6  |
| 1.6 Chain of Custody & Write Blocker . . . . .        | 6  |
| 1.7 Integrität . . . . .                              | 6  |
| 2 Dateisysteme . . . . .                              | 7  |
| 2.1 Linux-Dateisysteme . . . . .                      | 7  |
| 2.2 Windows-Dateisysteme . . . . .                    | 7  |
| 2.3 Apple-Dateisysteme . . . . .                      | 7  |
| 2.4 Weitere Dateisysteme . . . . .                    | 8  |
| 2.5 Wear Leveling & WAL-Dateien . . . . .             | 8  |
| 3 Partitionen & Bootloader . . . . .                  | 9  |
| 3.1 MBR (Master Boot Record) . . . . .                | 9  |
| 3.2 GPT (GUID Partition Table) . . . . .              | 9  |
| 3.3 mmls — Partitionstabelle interpretieren . . . . . | 10 |
| 4 Befehlsreferenz . . . . .                           | 11 |
| 4.1 Image-Handling . . . . .                          | 11 |
| 4.2 Sleuth Kit — 5 Kernbefehle . . . . .              | 11 |
| 4.3 Weitere TSK-Tools . . . . .                       | 12 |
| 4.4 Carving & Binäranalyse . . . . .                  | 12 |
| 4.5 Registry & Windows . . . . .                      | 13 |
| 4.6 E-Mail & Datenbanken . . . . .                    | 13 |
| 4.7 Timeline . . . . .                                | 13 |
| 4.8 Metadaten & Sonstiges . . . . .                   | 14 |
| 5 Standard-Workflow . . . . .                         | 15 |
| 5.1 Überblick . . . . .                               | 15 |
| 5.1.1 1. Integrität prüfen . . . . .                  | 15 |
| 5.1.2 2. Image mounten . . . . .                      | 15 |

|       |                                                |    |
|-------|------------------------------------------------|----|
| 5.1.3 | 3. Partitionstabelle analysieren . . . . .     | 15 |
| 5.1.4 | 4. Loop Device erstellen . . . . .             | 16 |
| 5.1.5 | 5. Dateisystem analysieren . . . . .           | 16 |
| 5.1.6 | 6. Dateien extrahieren . . . . .               | 16 |
| 5.1.7 | 7. Partition mounten (optional) . . . . .      | 16 |
| 5.1.8 | 8. Aufräumen (LIFO) . . . . .                  | 16 |
| 6     | Windows Registry . . . . .                     | 17 |
| 6.1   | Registry-Struktur . . . . .                    | 17 |
| 6.2   | Hives extrahieren . . . . .                    | 17 |
| 6.3   | Regripper-Plugins . . . . .                    | 18 |
| 6.4   | Windows-Login umgehen . . . . .                | 18 |
| 7     | Volume Shadow Copies . . . . .                 | 19 |
| 7.1   | Konzept . . . . .                              | 19 |
| 7.2   | Workflow . . . . .                             | 19 |
| 7.3   | Anwendungsfall: PST-Recovery . . . . .         | 19 |
| 7.4   | Aufräumen . . . . .                            | 20 |
| 8     | Timeline-Analyse . . . . .                     | 21 |
| 8.1   | TSK mactime . . . . .                          | 21 |
| 8.2   | Plaso / log2timeline . . . . .                 | 21 |
| 8.3   | Browser-History direkt per SQLite . . . . .    | 21 |
| 8.3.1 | Chrome . . . . .                               | 22 |
| 8.3.2 | Firefox . . . . .                              | 22 |
| 8.3.3 | Edge (ESE-Datenbank) . . . . .                 | 22 |
| 9     | Verschlüsselung . . . . .                      | 23 |
| 9.1   | BitLocker . . . . .                            | 23 |
| 9.1.1 | VMK aus SPI-Dump extrahieren . . . . .         | 23 |
| 9.1.2 | BitLocker-Volume entschlüsseln . . . . .       | 23 |
| 9.2   | dmsetup — Partitionen zusammenführen . . . . . | 23 |
| 9.3   | QEMU-Virtualisierung . . . . .                 | 24 |
| 9.4   | VeraCrypt / TrueCrypt . . . . .                | 24 |
| 9.5   | Aufräumen (LIFO) . . . . .                     | 24 |
| 10    | Malware-Analyse . . . . .                      | 25 |
| 10.1  | Prefetch-Analyse . . . . .                     | 25 |
| 10.2  | Autostart-Einträge . . . . .                   | 25 |
| 10.3  | YARA-Regeln . . . . .                          | 25 |
| 10.4  | Event Logs (EVTX) . . . . .                    | 26 |
| 10.5  | Weitere Indikatoren . . . . .                  | 26 |
| 11    | Netzwerk . . . . .                             | 27 |
| 11.1  | Port Knocking . . . . .                        | 27 |

|      |                                        |    |
|------|----------------------------------------|----|
| 11.2 | Forensisches Image erstellen . . . . . | 27 |
| 11.3 | Netzwerkadapter-Analyse . . . . .      | 28 |
| 11.4 | FUSE-Treiber . . . . .                 | 28 |
| 12   | Darknet & Tor . . . . .                | 29 |
| 12.1 | Tor Hidden Service . . . . .           | 29 |
| 12.2 | Tor-Artefakte suchen . . . . .         | 29 |
| 12.3 | SquashFS-Container . . . . .           | 29 |
| 12.4 | Versteckte ZIP-Archive . . . . .       | 30 |
| 13   | Mobile Forensik . . . . .              | 31 |
| 13.1 | Android-Images . . . . .               | 31 |
| 13.2 | SQLite-Datenbanken . . . . .           | 31 |
| 13.3 | EXIF/GPS-Daten . . . . .               | 31 |
| 13.4 | YAFFS2 / Nexus Nandump . . . . .       | 32 |

# 1 Grundlagen

## 1.1 Allocated vs. Unallocated Space

**Allocated Space** Aktiv genutzter Speicher mit Referenz im Dateisystem. Dateien sind über das Dateisystem direkt zugreifbar.

**Unallocated Space** Freigegebener Speicher — Daten physisch noch vorhanden, bis sie überschrieben werden. Forensisch besonders wertvoll, da gelöschte Dateien hier oft noch rekonstruierbar sind.

## 1.2 EWF-Format (E01)

Expert Witness Format — das Standard-Imageformat in der digitalen Forensik.

- Komprimiertes forensisches Image mit integrierten Metadaten (Case-Nr., Examiner, Datum)
- Integrierte Prüfsummen (MD5) zur Integritätssicherung
- Enthält sowohl Allocated als auch Unallocated Space (bitgenaue Kopie)
- Segmente werden bei 2 GB gesplittet (.E01–.E07)
- Metadaten einsehen mit `ewfinfo`, Integrität prüfen mit `ewfverify`

## 1.3 RAW → EWF → xmount Kreislauf

1. Originaldisk (z.B. 8 TB) wird mit `ewfacquire` als komprimiertes EWF-Image gesichert ( 4 TB)
2. `xmount` stellt das EWF als virtuelles RAW-Image (8 TB) bereit
3. Ein Overlay (`-cache`) speichert alle Änderungen — das Original bleibt unverändert

```
1 ewfacquire /dev/sdX          # Disk -> EWF
2 xmount --in ewf img.E0* --cache /tmp/img.ovl --out raw /ewf
3 # /ewf/img.dd ist jetzt ein virtuelles RAW-Image
```

## 1.4 Physische vs. logische Analyse

**Physische Analyse** Zugriff auf alles (Allocated + Unallocated). Über TSK-Tools wie `fls`, `icat`, `istat`. Ermöglicht Zugriff auf gelöschte Dateien und Dateifragmente.

**Logische Analyse** Image wird als Dateisystem gemountet (`mount -o ro`). Nur Allocated Space sichtbar — wie ein normales Dateisystem. Einfacher zu navigieren, aber weniger Informationen.

## 1.5 File Carving

Letzte Möglichkeit zur Datenrettung — Dateien werden anhand bekannter Header/Footer-Signaturen rekonstruiert, ohne Dateisystem-Metadaten.

- Tools: `foremost`, `scalpel`
- Verliert Dateinamen, Verzeichnisstruktur und Metadaten
- Funktioniert auch bei beschädigten oder formatierten Dateisystemen
- Nur sinnvoll, wenn `tsk_recover` keine Ergebnisse liefert

## 1.6 Chain of Custody & Write Blocker

**Chain of Custody** Lückenlose Dokumentation aller Aktionen an Beweismitteln. Jede Analyse, jeder Zugriff muss protokolliert werden, um die Beweiskraft zu erhalten.

**Write Blocker** Hardware oder Software, die Schreibzugriffe auf Beweismittel verhindert. EWF-Images sind inhärent read-only. Bei `xmount` sorgt der Overlay-Cache dafür, dass das Original unverändert bleibt.

## 1.7 Integrität

Immer als ersten Schritt die Integrität prüfen — vor und nach der Analyse.

```
1 ewfverify image.E01           # Interne EWF-Pruefsummen pruefen
2 md5sum image.E01 image.E02    # MD5-Hashes berechnen
3 md5sum -c md5s.txt            # Gegen gespeicherte Hashes vergleichen
```

## 2 Dateisysteme

### 2.1 Linux-Dateisysteme

**Ext2** Ohne Journal — einfach, aber bei Absturz anfällig für Inkonsistenzen.

**Ext3** Mit Journal — protokolliert Änderungen vor dem Schreiben.

**Ext4** Aktueller Standard. Blockgruppen, Inodes, Extents. Vollständig von TSK (Sleuth Kit) unterstützt.

**XFS** Experimentell in aktuellen Linux-Versionen (ab Kernel v13).

### 2.2 Windows-Dateisysteme

**NTFS** New Technology File System — Standard für Windows. Master File Table (MFT) als zentrale Verwaltungsstruktur. Unterstützt Alternate Data Streams (ADS), Prefetch-Dateien und Journaling. Vollständig von TSK unterstützt.

**FAT32 / exFAT** Einfaches Dateisystem ohne Berechtigungen. Häufig auf USB-Sticks und SD-Karten.

### 2.3 Apple-Dateisysteme

**APFS** Apple File System (ab macOS 10.13) — verschlüsselt, Copy-on-Write, Snapshots.

**HFS / HFS+** Hierarchical File System — ältere Mac-Systeme.

## 2.4 Weitere Dateisysteme

**YAFFS2** Yet Another Flash File System 2 — für alte Handys und IoT-Geräte. Nicht logisch analysierbar (kein `mount` möglich), nur physisch mit `fls`.

```
1 fls -pr nexus.nandump | grep '\.db$'
2 icat nexus.nandump 524 > /tmp/CachedPosition.db
3 sqlitebrowser /tmp/CachedPosition.db
```

**UFS** Unix File System — von FreeBSD und OpenBSD genutzt.

**SquashFS** Komprimiertes, read-only Dateisystem. Wird als Container für versteckte Daten genutzt. Erkennung mit `binwalk`, Extraktion mit `unsquashfs`.

```
1 binwalk image.dd # SquashFS-Offset finden
2 dd if=image.dd skip=OFFSET bs=1 count=SIZE of=/tmp/sq.img
3 unsquashfs /tmp/sq.img # Inhalt extrahieren
```

## 2.5 Wear Leveling & WAL-Dateien

Flash-Speicher (SSDs, SD-Karten, Handys) verteilt Schreibvorgänge gleichmäßig über alle Zellen (**Wear Leveling**).

- Konsequenz: Gelöschte Daten werden seltener überschrieben als bei HDDs
- SQLite-Datenbanken nutzen **WAL-Dateien** (Write-Ahead Log, `.db-wal`) und **SHM-Dateien** (`.db-shm`)
- WAL kann aktuellere Daten enthalten als die `.db`-Datei selbst
- Autopsy liest WAL-Dateien automatisch mit — bei manueller Analyse beide extrahieren
- Relevant für WhatsApp (`wa.db-wal`, `msgstore.db-wal`) und andere Apps



## 3 Partitionen & Bootloader

### 3.1 MBR (Master Boot Record)

Erste 512 Bytes der Festplatte — enthält Bootloader und Partitionstabelle.

- **446 Bytes:** Bootloader-Code
- **4 × 16 Bytes:** Partitionseinträge (maximal 4 primäre Partitionen)
- **2 Bytes:** Signatur 0x55AA (Boot-Signatur am Ende)
- Maximale Festplattengröße: 2 TB (bzw. 3,2 TB bei 512-Byte-Sektoren)

MBR im Hex-Editor ansehen:

```
1 dd if=/dev/sda count=1 | xxd
```

### 3.2 GPT (GUID Partition Table)

Nachfolger von MBR mit wesentlichen Verbesserungen.

- Unterstützt mehr als 4 Partitionen (wichtig für mobile Geräte, z.B. Android)
- Keine 2-TB-Grenze
- Redundante Kopie der Partitionstabelle am Ende der Festplatte
- Enthält Protective MBR für Abwärtskompatibilität

### 3.3 mmls — Partitionstabelle interpretieren

`mmls` zeigt die Partitionstabelle eines Images. Die Ausgabe liefert den **Offset in Sektoren**, der für alle TSK-Tools als `-o` Parameter benötigt wird.

```
1 mmls image.E01
```

Beispielausgabe:

|   | Slot      | Start      | End        | Length     | Description          |
|---|-----------|------------|------------|------------|----------------------|
| 2 | 000: Meta | 0000000000 | 0000000000 | 0000000001 | Primary Table (#0)   |
| 3 | 001: ---  | 0000000001 | 0000002047 | 0000002047 | Unallocated          |
| 4 | 002: 000  | 0000002048 | 0000206847 | 0000204800 | EFI system partition |
| 5 | 003: 001  | 0000206848 | 0248655871 | 0248449024 | Basic data partition |

**Start** Offset in Sektoren — diesen Wert als `-o` bei TSK-Tools verwenden

**Length** Länge in Sektoren —  $\text{Length} \times 512 = \text{Partitionsgröße in Bytes}$

**DOS** Partitionstabellen-Typ ist MBR

**GPT** Partitionstabellen-Typ ist GPT

**Meta** Metadaten-Einträge (Tabelle selbst)

— Unallocated Space zwischen Partitionen

Beispiel: Dateisystem der dritten Partition analysieren:

```
1 fsstat -o 206848 image.E01
2 fls -pr -o 206848 image.E01
```

## 4 Befehlsreferenz

### 4.1 Image-Handling

| Befehl     | Syntax                                                 | Beschreibung                                   |
|------------|--------------------------------------------------------|------------------------------------------------|
| ewfverify  | ewfverify image.E01                                    | EWF-Prüfsummen verifizieren                    |
| ewfinfo    | ewfinfo image.E01                                      | EWF-Metadaten anzeigen (Case, Datum, Examiner) |
| ewfmount   | ewfmount image.E01 /mnt/ewf                            | EWF als RAW-Device mounten                     |
| ewfacquire | ewfacquire /dev/sdX                                    | Forensisches EWF-Image erstellen               |
| xmount     | xmount -in ewf img.E0* -cache /tmp/x.ovl -out raw /ewf | EWF→RAW konvertieren mit Overlay               |
| losetup    | losetup -partscan -find -show /ewf/img.dd              | Loop Device für Partitionszugriff              |

### 4.2 Sleuth Kit — 5 Kernbefehle

| Befehl | Syntax                               | Beschreibung                                         |
|--------|--------------------------------------|------------------------------------------------------|
| mmls   | mmls image.E01                       | Partitionstabelle anzeigen                           |
| fsstat | fsstat -o OFFSET image.E01           | Dateisystem-Infos (Typ, Blockgröße)                  |
| fls    | fls -pr -o OFFSET image.E01          | Dateien auflisten (-p=Pfad, -r=rekursiv, -d=deleted) |
| icat   | icat -o OFFSET image.E01 INODE > out | Datei per Inode extrahieren                          |
| istat  | istat -o OFFSET image.E01 INODE      | Inode-Metadaten (Timestamps, Größe, Blöcke)          |

### 4.3 Weitere TSK-Tools

| Befehl      | Syntax                                  | Beschreibung                       |
|-------------|-----------------------------------------|------------------------------------|
| tsk_recover | tsk_recover -o OFFSET image.E01 outdir/ | Gelöschte Dateien wiederherstellen |
| ifind       | ifind -o OFFSET image.E01 -d BLOCK      | Inode zu Block finden              |
| ffind       | ffind -o OFFSET image.E01 INODE         | Dateiname zu Inode finden          |

### 4.4 Carving & Binäranalyse

| Befehl   | Syntax                               | Beschreibung                            |
|----------|--------------------------------------|-----------------------------------------|
| foremost | foremost -o output/ -i image.dd      | File Carving nach Header/Footer         |
| scalpel  | scalpel -o output/ image.dd          | File Carving (konfigurierbar)           |
| binwalk  | binwalk image.dd                     | Eingebettete Dateien finden             |
| binwalk  | binwalk -E file                      | Entropie-Analyse (hoch = verschlüsselt) |
| dd       | dd if=src of=dst bs=1 skip=X count=Y | Daten blockweise kopieren               |
| strings  | strings file   grep keyword          | Lesbare Zeichenketten suchen            |
| file     | file recovered.dat                   | Dateityp identifizieren                 |
| xxd      | xxd file   head                      | Hex-Dump anzeigen                       |

## 4.5 Registry & Windows

| Befehl       | Syntax                                   | Beschreibung                         |
|--------------|------------------------------------------|--------------------------------------|
| regripper    | regripper -p PLUGIN -r HIVE              | Registry-Hive mit Plugin analysieren |
| sccainfo     | sccainfo file.pf                         | Prefetch-Datei analysieren           |
| chntpw       | chntpw -i SAM                            | Windows-Passwort zurücksetzen        |
| dislocker    | dislocker /dev/loopXpY -K key.vmk - /mnt | BitLocker-Volume entschlüsseln       |
| vshadowinfo  | vshadowinfo /dev/loopXpY                 | Volume Shadow Copies auflisten       |
| vshadowmount | vshadowmount /dev/loopXpY /mnt/vss       | Shadow Copies mounten                |

## 4.6 E-Mail & Datenbanken

| Befehl        | Syntax                         | Beschreibung                     |
|---------------|--------------------------------|----------------------------------|
| readpst       | readpst -o outdir/ file.pst    | Outlook PST zu mbox konvertieren |
| pffexport     | pffexport -t outdir/ file.pst  | PST detailliert exportieren      |
| sqlite3       | sqlite3 db.sqlite SSELECT ..." | SQLite-Datenbank abfragen        |
| sqlitebrowser | sqlitebrowser file.db          | SQLite GUI                       |

## 4.7 Timeline

| Befehl          | Syntax                                   | Beschreibung                       |
|-----------------|------------------------------------------|------------------------------------|
| log2timeline.py | log2timeline.py out.plaso /mnt           | Plaso Super-Timeline erstellen     |
| psort.py        | psort.py -o l2tcsv -w out.csv file.plaso | Plaso-Timeline als CSV exportieren |
| mactime         | mactime -b body.txt > timeline.csv       | TSK Body File → Timeline           |

## 4.8 Metadaten & Sonstiges

| Befehl   | Syntax                                            | Beschreibung                       |
|----------|---------------------------------------------------|------------------------------------|
| exiftool | exiftool -csv -GPSLatitude<br>-GPSLongitude *.jpg | EXIF/GPS-Daten auslesen            |
| md5sum   | md5sum file                                       | MD5-Prüfsumme berechnen            |
| yara     | yara rules.yar file                               | YARA-Regel auf Datei an-<br>wenden |

## 5 Standard-Workflow

### 5.1 Überblick

Der Standard-Workflow für die forensische Image-Analyse folgt immer demselben Schema: Integrität prüfen → Image mounten → Partitionen analysieren → Dateien extrahieren → Aufräumen (LIFO).

#### 5.1.1 1. Integrität prüfen

Vor jeder Analyse müssen die Prüfsummen des Images verifiziert werden. Der gespeicherte MD5-Hash muss mit dem berechneten übereinstimmen.

```
1 ewfverify image.E01
2 md5sum image.E01 image.E02 ...
```

#### 5.1.2 2. Image mounten

EWf-Image als virtuelles RAW-Device mit Overlay bereitstellen. Der Parameter `-cache` ist essenziell: alle Änderungen werden im Overlay gespeichert, das Original bleibt unverändert.

```
1 mkdir -p /tmp/ewf
2 xmount --in ewf image.E0* --cache /tmp/img.ovl --out raw /tmp/ewf
```

#### 5.1.3 3. Partitionstabelle analysieren

```
1 mmls /tmp/ewf/image.dd
```

Die Ausgabe zeigt das Partitionslayout. Der **Start**-Offset jeder Partition wird als `-o` Parameter für alle TSK-Tools benötigt.

#### 5.1.4 4. Loop Device erstellen

```
1 losetup --partscan --find --show /tmp/ewf/image.dd
```

Erstellt `/dev/loopXpN` für jede Partition automatisch. Alternative für eine einzelne Partition:

```
1 losetup -o $((OFFSET*512)) /dev/loop0 /tmp/ewf/image.dd
```

#### 5.1.5 5. Dateisystem analysieren

```
1 fsstat /dev/loopXpN | head
2 fls -pr /dev/loopXpN
3 fls -r -d /dev/loopXpN    # nur gelöschte Dateien
```

- `fsstat` — zeigt Dateisystemtyp, Version, Blockgröße
- `fls -pr` — listet alle Dateien rekursiv mit vollem Pfad
- Gelöschte Dateien sind mit `*` markiert

#### 5.1.6 6. Dateien extrahieren

```
1 fls -pr /dev/loopXpN | grep -i 'filename'
2 icat /dev/loopXpN INODE > /tmp/recovered_file
3 file /tmp/recovered_file
```

Nach jeder Extraktion **immer** mit `file` verifizieren.

#### 5.1.7 7. Partition mounten (optional)

```
1 mount -o ro /dev/loopXpN /mnt
2 # oder mit Offset:
3 mount -o ro,loop,offset=$((OFFSET * 512)) /tmp/ewf/image.dd /mnt
```

Read-Only-Mount zum Browsen. Zeigt nur Allocated Space (logische Analyse).

#### 5.1.8 8. Aufräumen (LIFO)

Umgekehrte Reihenfolge — zuletzt gemountet wird zuerst unmountet:

```
1 umount /mnt
2 losetup -D
3 umount /tmp/ewf
```



## 6 Windows Registry

### 6.1 Registry-Struktur

Die Windows Registry besteht aus zwei Hauptkategorien:

**HKLM (HKEY\_LOCAL\_MACHINE)** — Systemweite Einstellungen

- **SAM** — Benutzerkonten, Passwort-Hashes
- **SYSTEM** — Hardwarekonfiguration, USB-Historie, Boot-Konfiguration
- **SOFTWARE** — Installierte Software, OS-Version
- Speicherort: Windows/System32/config/

**HKCU (HKEY\_CURRENT\_USER)** — Benutzerspezifische Einstellungen

- **NTUSER.DAT** — Benutzereinstellungen, zuletzt geöffnete Dokumente
- Speicherort: Users/<Benutzername>/NTUSER.DAT

### 6.2 Hives extrahieren

```
1 fls -pr /dev/loopXpN | grep 'SAM$'
2 icat /dev/loopXpN <INODE_SAM> > /tmp/SAM
3
4 fls -pr /dev/loopXpN | grep 'SYSTEM$'
5 icat /dev/loopXpN <INODE_SYSTEM> > /tmp/SYSTEM
6
7 fls -pr /dev/loopXpN | grep 'SOFTWARE$'
8 icat /dev/loopXpN <INODE_SOFTWARE> > /tmp/SOFTWARE
9
10 fls -pr /dev/loopXpN | grep 'NTUSER.DAT$'
11 icat /dev/loopXpN <INODE_NTUSER> > /tmp/NTUSER.DAT
```

Nach jeder Extraktion mit `file /tmp/HIVE` verifizieren.

### 6.3 Regripper-Plugins

| Plugin     | Hive                  | Information                                |
|------------|-----------------------|--------------------------------------------|
| winver     | SOFTWARE              | Windows-Version, Build-Nr., Install-Datum  |
| usbdevices | SYSTEM                | USB-Gerätenamen, VID/PID                   |
| usbstor    | SYSTEM                | USB-Timestamps (first/last), Seriennummern |
| usbstor2   | SYSTEM                | Erweiterte USB-Timestamps                  |
| devclass   | SYSTEM                | Geräteklassen inkl. Hersteller             |
| mountdev   | SYSTEM                | Laufwerksbuchstaben der USB-Geräte         |
| nic2       | SYSTEM                | Netzwerkadapter, DHCP-Server, IP-Adressen  |
| run        | SOFTWARE / NTUSER.DAT | Autostart-Einträge                         |
| recentdocs | NTUSER.DAT            | Zuletzt geöffnete Dokumente                |
| shellbags  | NTUSER.DAT            | Geöffnete Ordner (Explorer-Verlauf)        |
| uninstall  | SOFTWARE              | Installierte/deinstallierte Programme      |

```

1 # Plugin suchen
2 regripper -l | grep -B 1 USB
3
4 # Plugin ausführen
5 regripper -p winver -r SOFTWARE
6 regripper -p usbdevices -r SYSTEM
7 regripper -p usbstor -r SYSTEM
8 regripper -p nic2 -r SYSTEM

```

**Hinweis:** Windows 11 meldet sich in der Registry weiterhin als “Windows 10” — Microsoft hat die Versionsnummer nicht mehr erhöht.

### 6.4 Windows-Login umgehen

**Utility Manager Trick** utilman.exe durch cmd.exe ersetzen. Beim Login-Screen den Utility Manager öffnen → Shell-Zugang.

**chntpw** Interaktives Passwort-Reset über die SAM-Datei:

```

1 chntpw -i /tmp/SAM

```

## 7 Volume Shadow Copies

### 7.1 Konzept

- Windows erstellt automatisch Schattenkopien vor Updates und Neustarts
- Byte-für-Byte Kopie des Volumes zu einem bestimmten Zeitpunkt
- **Windows 7:** Wiederherstellung per CD
- **Windows 10:** Standardmäßig verfügbar
- **Windows 11:** Standardmäßig deaktiviert
- Forensisch wertvoll: gelöschte Dateien können in älteren Schattenkopien noch existieren

### 7.2 Workflow

```
1 # Shadow Copies auflisten
2 vshadowinfo /dev/loopXpN
3 # Ausgabe: Anzahl Stores, Timestamps, GUIDs
4
5 # Alle Stores mounten
6 mkdir -p /tmp/vshadow
7 vshadowmount /dev/loopXpN /tmp/vshadow
8 ls /tmp/vshadow/
9 # -> vss1, vss2, vss3, ... (jeder Store = ein Zeitpunkt)
10
11 # Inhalte durchsuchen
12 fls -pr /tmp/vshadow/vss1 | grep -i 'gesuchte_datei'
13
14 # Datei aus Shadow Copy extrahieren
15 icat /tmp/vshadow/vss3 <INODE> > /tmp/recovered_file
```

### 7.3 Anwendungsfall: PST-Recovery

Gelöschte Outlook-PST-Dateien über Shadow Copies wiederherstellen:

```
1 # PST in allen Shadow Copies suchen
2 for i in /tmp/vshadow/vss*; do
3     echo "=== $i ==="
4     fls -pr "$i" | grep -i '\.pst'
5 done
6
7 # PST extrahieren
8 icat /tmp/vshadow/vssN <INODE> > /tmp/recovered.pst
9
10 # PST analysieren
11 readpst -o /tmp/pst_out /tmp/recovered.pst
12 cat /tmp/pst_out/*.mbox | less
```

Alternativer Viewer mit mehr Details:

```
1 pffinfo /tmp/recovered.pst
2 pffexport /tmp/recovered.pst
```

## 7.4 Aufräumen

```
1 umount /tmp/vshadow
```

## 8 Timeline-Analyse

### 8.1 TSK mactime

Einfache Timeline aus Sleuth Kit Body File:

```
1 # Body File erzeugen (alle Dateien mit Timestamps)
2 fls -r -m "/" -o OFFSET image.E01 > body.txt
3
4 # Timeline generieren
5 mactime -b body.txt > timeline.csv
```

**MAC-Timestamps** Modified, Accessed, Changed

**Body File Format** Enthält Inode, Permissions, UID, GID, Größe und alle Timestamps

### 8.2 Plaso / log2timeline

Mächtigere Super-Timeline — analysiert viele Artefakttypen über reine Datei-Timestamps hinaus:

```
1 # Timeline aus gemounteter Partition erstellen
2 log2timeline.py /tmp/timeline.plaso /mnt
3
4 # Nur bestimmte Parser verwenden (schneller)
5 log2timeline.py --parsers "chrome_history,firefox_history" \
6     /tmp/browser.plaso /mnt
7
8 # Als CSV exportieren
9 psort.py -o l2tcsv -w /tmp/timeline.csv /tmp/timeline.plaso
```

Verfügbare Parser für Web-History:

- chrome\_history, firefox\_history, msedge\_history
- ie\_webcache, safari\_history, opera\_history

### 8.3 Browser-History direkt per SQLite

Alternative: Datenbanken direkt extrahieren und abfragen.

### 8.3.1 Chrome

```
1 fls -pr /dev/loopXpN | grep -i 'History$' | grep -i chrome
2 icat /dev/loopXpN <INODE> > /tmp/chrome_history.db
3 sqlite3 /tmp/chrome_history.db \
4 "SELECT url, title, visit_count, last_visit_time
5 FROM urls ORDER BY last_visit_time DESC LIMIT 50;"
```

### 8.3.2 Firefox

```
1 fls -pr /dev/loopXpN | grep -i 'places.sqlite'
2 icat /dev/loopXpN <INODE> > /tmp/places.sqlite
3 sqlite3 /tmp/places.sqlite \
4 "SELECT url, title, visit_count, last_visit_date
5 FROM moz_places ORDER BY last_visit_date DESC LIMIT 50;"
```

### 8.3.3 Edge (ESE-Datenbank)

```
1 fls -pr /dev/loopXpN | grep -i 'WebCacheV01'
2 icat /dev/loopXpN <INODE> > /tmp/WebCacheV01.dat
3 esedbexport -t /tmp/edge_cache /tmp/WebCacheV01.dat
```

## 9 Verschlüsselung

### 9.1 BitLocker

Windows-Festplattenverschlüsselung. Zum Entschlüsseln wird der Volume Master Key (VMK) benötigt.

#### 9.1.1 VMK aus SPI-Dump extrahieren

Hardware-Angriff über Logic-Analyzer-Daten:

```
1 # YAFFS in Firmware suchen
2 binwalk firmware.bin | grep -i yaffs
3
4 # VMK mit ARNE-Tool extrahieren
5 ./arne -i digital.csv \
6   -k "1=SCLK,2=CS,3=MOSI,4=MISO" -o key.vmk
7 xxd key.vmk
```

#### 9.1.2 BitLocker-Volume entschlüsseln

```
1 mkdir -p /tmp/bitlocker /tmp/windows
2
3 # Mit VMK-Key entschlüsseln
4 dislocker /dev/loopXpN -K key.vmk -- /tmp/bitlocker
5
6 # NTFS dirty flag beheben
7 ntfsfix -d /tmp/bitlocker/dislocker-file
8
9 # Entschlüsselte Partition loopen
10 losetup --partscan --find --show \
11   /tmp/bitlocker/dislocker-file
```

### 9.2 dmsetup — Partitionen zusammenführen

Nach der Entschlüsselung müssen die Partitionen zu einem virtuellen Gesamtdisk zusammengeführt werden:

```
1 # dmsetup.txt (Loop-Nummern anpassen):  
2 # 0 239616 linear /dev/loop37 0  
3 # 239616 248416256 linear /dev/loop38 0  
4 # 248655872 1411072 linear /dev/loop37 248655872  
5  
6 cat dmsetup.txt | dmsetup create merged
```

### 9.3 QEMU-Virtualisierung

Entschlüsseltes Image als VM starten:

```
1 qemu-system-x86_64 -bios /usr/share/ovmf/OVMF.fd \  
2 -cpu Skylake-Client-v3 -enable-kvm -usb \  
3 -hda /dev/mapper/merged -smp 4 -m 4096
```

### 9.4 VeraCrypt / TrueCrypt

Container erkennen durch hohe Entropie:

```
1 binwalk -E verdaechtige_datei  
2 # Hohe Entropie (>7.5) = wahrscheinlich verschlüsselt
```

Container öffnen (falls Passwort bekannt):

```
1 veracrypt --text --mount container.vc /mnt/container \  
2 --password="PASSWORT" --non-interactive
```

### 9.5 Aufräumen (LIFO)

```
1 dmsetup remove merged  
2 losetup -d /dev/loop38  
3 umount /tmp/bitlocker  
4 losetup -d /dev/loop37  
5 umount /tmp/ewf
```



## 10 Malware-Analyse

### 10.1 Prefetch-Analyse

Windows Prefetch: Dateien im Verzeichnis `Windows/Prefetch/` mit Endung `.pf`. Enthalten Informationen über ausgeführte Programme (Name, Run Count, letzte Ausführungszeit). Prefetch lädt häufig verwendete DLLs vor — zeigt auch Autostart-Malware.

```
1 # Prefetch-Dateien finden
2 fls -pr /dev/loopXpN | grep -i prefetch
3 mount -o ro /dev/loopXpN /mnt
4 find /mnt -type f -iname "*.pf" -print0 | xargs -0 ls -la
5
6 # Prefetch analysieren
7 sccainfo /mnt/Windows/Prefetch/PROGRAMM.EXE-HASH.pf
8 # Ausgabe: Run Count, Last Run Time(s)
```

### 10.2 Autostart-Einträge

Registry Run-Keys zeigen Programme, die bei jedem Login starten:

```
1 regripper -p run -r /tmp/SOFTWARE
2 regripper -p run -r /tmp/NTUSER.DAT
```

Geplante Tasks:

```
1 fls -pr /dev/loopXpN | grep -i 'Tasks'
2 icat /dev/loopXpN <INODE_TASK> > /tmp/task.xml
3 cat /tmp/task.xml
```

### 10.3 YARA-Regeln

Pattern-Matching für Malware-Erkennung. Regelformat:

```
1 rule rule_name {
2     strings:
3         $var1 = "string" nocase
4         $var2 = "string2" nocase
5     condition:
6         $var1 and $var2
7 }
```

Beispiel — UPX-gepackte Binaries erkennen:

```
1 rule upx_find {
2   strings:
3     $upx0 = "upx0" nocase
4     $upx1 = "upx1" nocase
5     $rsrc = ".rsrc" nocase
6   condition:
7     $upx0 and $upx1 and $rsrc
8 }
```

Beispiel — Gestohlene Signaturen (Stuxnet-Referenz):

```
1 rule stolen_signatures {
2   strings:
3     $find1 = "realtek" nocase
4     $find2 = "jmicron" nocase
5   condition:
6     $find1 or $find2
7 }
```

Anwendung: `yara rules.yar suspicious_file`

## 10.4 Event Logs (EVTX)

Windows Event Logs für Security-Analyse:

```
1 fls -pr /dev/loopXpN | grep -i '\.evtx$'
2 icat /dev/loopXpN <INODE> > /tmp/security.evtx
3 evtx_dump.py /tmp/security.evtx | grep -i "malware\|virus\|threat"
```

## 10.5 Weitere Indikatoren

**Hosts-Datei:** DNS-Hijacking erkennen (`icat` → `hosts` prüfen)

**Strings-Analyse:** Verdächtige Binaries nach Schlüsselwörtern durchsuchen:

```
1 icat /dev/loopXpN <INODE_EXE> > /tmp/suspicious.exe
2 strings /tmp/suspicious.exe | grep -Ei "http|cmd|powershell|download|bitcoin"
```

## 11 Netzwerk

### 11.1 Port Knocking

Technik um versteckte Dienste zu finden. Ports in bestimmter Reihenfolge ansprechen:

```
1 # Einzelne Ports scannen
2 nmap -p PORT target_ip
3
4 # Port Knocking mit hping3
5 hping3 -S target_ip -p PORT1
6 hping3 -S target_ip -p PORT2
7 hping3 -S target_ip -p PORT3
```

TCP-Flags in der Antwort:

**RA (Reset+ACK):** Port geschlossen

**SA (SYN+ACK):** Port offen

### 11.2 Forensisches Image erstellen

Ablauf: leeres Image erstellen → partitionieren → formatieren → Daten kopieren → EWF erzeugen:

```
1 # Leeres Image erstellen
2 dd if=/dev/zero of=usb.img bs=1M count=512
3
4 # Partitionieren
5 fdisk usb.img
6 # -> n (new), p (primary), Typ aendern: t, 07 (NTFS)
7
8 # Loop Device und formatieren
9 losetup --partscan --find --show usb.img
10 mkfs.ntfs /dev/loop0p1
11
12 # Mounten und Daten kopieren
13 mount /dev/loop0p1 /mnt
14 rsync -av source/ /mnt/
15
16 # EWF-Image erstellen
17 ewfacquire /dev/loop0
18 # Case-Nummer, Examiner, Beschreibung angeben
```

### 11.3 Netzwerkadapter-Analyse

Über Windows Registry:

```
1 regripper -p nic2 -r /tmp/SYSTEM
```

Zeigt: Adapter-Name, DHCP-Server, IP-Adresse, Subnetzmaske, Gateway, DNS. Forensisch relevant: verbundene Netzwerke (z.B. Fritz.box → Heimnetzwerk).

### 11.4 FUSE-Treiber

Filesystem in Userspace — ermöglicht Zugriff auf verschiedene Dateisysteme:

- `ntfs-3g` — NTFS lesen/schreiben
- `xmount` — FUSE-basiert für Image-Konvertierung

## 12 Darknet & Tor

### 12.1 Tor Hidden Service

Konfiguration in /etc/tor/torrc:

```
1 # torrc Konfiguration
2 HiddenServiceDir /var/lib/tor/hidden_service/
3 HiddenServicePort 80 127.0.0.1:80
4
5 # Service starten
6 systemctl restart tor
7
8 # .onion-Adresse auslesen
9 cat /var/lib/tor/hidden_service/hostname
```

### 12.2 Tor-Artefakte suchen

Im forensischen Image nach Tor-Spuren suchen:

```
1 # Tor-Dateien im Dateisystem
2 fls -pr /dev/loopXpN | grep -i 'tor'
3
4 # Prefetch: Wurde Tor ausgefuehrt?
5 find /mnt -type f -iname "*.pf" | grep -i tor
6 sccainfo /mnt/Windows/Prefetch/TOR.EXE-*.pf
7
8 # Onion-URLs in Browser-History (Firefox-basiert)
9 fls -pr /dev/loopXpN | grep -i 'places.sqlite' | grep -i tor
10 icat /dev/loopXpN <INODE> > /tmp/tor_places.sqlite
11 sqlite3 /tmp/tor_places.sqlite \
12     "SELECT url FROM moz_places WHERE url LIKE '%.onion%';"
13
14 # Strings-Suche nach .onion-Adressen
15 strings /dev/loopXpN | grep '\.onion'
16 grep -r "\.onion" /mnt/Users/ 2>/dev/null
```

### 12.3 SquashFS-Container

Versteckte Daten in komprimierten Containern am Ende eines Images:

```
1 # Container finden
2 binwalk image.dd
3 # -> Squashfs filesystem at offset X
4
5 # Container extrahieren
6 dd if=image.dd bs=1 skip=OFFSET count=SIZE \
7    of=/tmp/container.squashfs
8
9 # Entpacken
10 unsquashfs /tmp/container.squashfs
11 ls squashfs-root/
```

Beispiel: mkp224o (Tor Vanity Address Generator) — generiert .onion-Adressen mit bestimmten Präfixen. Git-Repos im Container können Aktivitäten dokumentieren.

## 12.4 Versteckte ZIP-Archive

In dd-Images eingebettete Archive finden:

```
1 binwalk image.dd
2 # -> Zip archive at offset X, size Y
3
4 dd if=image.dd skip=OFFSET bs=1 count=SIZE \
5    of=/tmp/hidden.zip
6 unzip /tmp/hidden.zip
```

## 13 Mobile Forensik

### 13.1 Android-Images

Android nutzt GPT mit vielen Partitionen. Wichtigste Partition: USERDATA (enthält App-Daten, Fotos, Datenbanken).

```
1 mmls android.E01
2 # Falls Speicherzugriffsfehler:
3 xmount --in ewf android.E01 \
4   --cache /tmp/phone.ovl --out raw /tmp/phone
5 mmls /tmp/phone/android.dd
6
7 losetup --partscan --find --show /tmp/phone/android.dd
8 ll /dev/loop*
```

### 13.2 SQLite-Datenbanken

Android speichert fast alles in SQLite: Kontakte, SMS, Anruflisten, Browser-History, GPS.

```
1 # Alle Datenbanken finden
2 fls -pr /dev/loopXpN | grep '\.db$'
3
4 # Nur nicht-geloeschte Datenbanken
5 fls -pr /dev/loopXpN | grep '\.db$' | grep -v '*'
6
7 # Datenbank extrahieren und oeffnen
8 icat /dev/loopXpN <INODE> > /tmp/database.db
9 file /tmp/database.db
10 sqlitebrowser /tmp/database.db
11 # oder:
12 sqlite3 /tmp/database.db ".tables"
```

Wichtig: WAL-Dateien (`.db-wal`) können aktuellere Daten enthalten als die Hauptdatenbank. Autopsy liest WAL automatisch mit.

### 13.3 EXIF/GPS-Daten

Fotos enthalten oft GPS-Koordinaten in EXIF-Metadaten:

```

1 # Alle Bilder mit GPS-Daten auslesen
2 exiftool -csv -GPSLatitude -GPSLongitude \
3   -GPSLatitudeRef -GPSLongitudeRef \
4   -DateTimeOriginal /tmp/photos/*.jpg > /tmp/gps_data.csv
5
6 # Einzelbild pruefen
7 exiftool photo.jpg | grep -E "GPS|Date|Time"

```

DMS zu Dezimalgrad umrechnen:

$$48^{\circ} 4' 35.00'' \text{ N} = 48 + \frac{4}{60} + \frac{35}{3600} = 48.0764^{\circ}$$

Fahrtroute rekonstruieren:

1. GPS-Daten nach `DateTimeOriginal` sortieren
2. Koordinaten in Dezimalgrad umrechnen
3. Chronologisch in Karte einzeichnen

Optional GPX-Track erzeugen:

```

1 exiftool -p gpx.fmt /tmp/photos/*.jpg > /tmp/track.gpx

```

## 13.4 YAFFS2 / Nexus Nandump

Alte Android-Geräte (z.B. Nexus) nutzen YAFFS2. Kein `mount` möglich, nur physische Analyse:

```

1 # Dateisystem-Typ pruefen
2 fsstat nexus.nandump # -> YAFFS
3
4 # Datenbanken suchen
5 fls -pr nexus.nandump | grep '\.db$' | grep -v '*'
6
7 # Geodaten extrahieren
8 icat nexus.nandump <INODE> > /tmp/CachedPosition.db
9 sqlitebrowser /tmp/CachedPosition.db

```